

OC4-1: Blind-Spot Decomposition Theory (BSDT) — Research Contribution

1. Executive Summary

Blind-Spot Decomposition Theory (BSDT) is an independently developed analytical framework for diagnosing why an AI fraud-detection system misses the specific fraudulent transactions it fails to catch. Conventional performance measures — such as accuracy, precision, or recall — tell you how often a system fails, but not why. BSDT breaks each failure down into one of four distinct, measurable causes, then produces a single correction score (MFLS) that can be applied on top of any existing detection system without rebuilding it. BSDT was developed entirely independently: no university affiliation, no co-authors, and no external funding.

2. Problem Statement

AI fraud-detection systems do not fail at random. When they miss a fraudulent transaction, there is typically a specific, identifiable reason. Four distinct failure patterns have been identified:

- Camouflage: the fraudulent transaction looks so similar to normal, legitimate activity that the detection system cannot tell them apart — like a skilled forger whose work passes automated signature checks.
- Information gap: the detection system does not have access to the type of information that would reveal the fraud. It is reaching a judgment while blind to evidence that might change its conclusion.
- Unseen pattern: the fraudulent activity looks nothing like the fraud examples the system was trained on. Because the system has never encountered this pattern before, it has no basis to flag it.
- Outdated baseline: the nature of fraud has changed since the detection system was last updated. What used to be unusual behaviour may now look ordinary — and vice versa — so the system's understanding of 'suspicious' no longer reflects current conditions.

Conventional evaluation tools merge all four failure modes into a single aggregate accuracy figure. BSDT separates them, making each one independently measurable — and enables a targeted correction to be applied on top of an existing system without rebuilding it from scratch.

3. BSDT Framework — Four-Channel Decomposition

The framework analyses each transaction through four independent channels. Each channel produces a score between 0 and 1 (higher = greater blind-spot risk for that cause). The four scores are then averaged to produce the final correction value:

- Channel 1 — Camouflage (δ_C): measures how closely the transaction resembles known fraud patterns. A score near 1 indicates the transaction looks suspicious to an informed examiner; a score near 0 means it blends in with normal activity.
- Channel 2 — Information Gap (δ_G): measures how much of the relevant evidence the system is missing. A high score indicates the system is making a decision with incomplete information.
- Channel 3 — Unseen Pattern (δ_A): measures how different this transaction's behaviour is from anything the system learned from during training. A high score means the system has no reliable reference for this type of activity.
- Channel 4 — Outdated Baseline (δ_T): measures how much the general pattern of transactions has changed since the system was last calibrated. A high score indicates the system's reference point may no longer reflect current conditions.

Formal definitions (each channel produces a score bounded between 0 and 1; the four scores are combined equally to form the final MFLS correction value):

Table E1: The mathematical definition of each channel. Each score (δ) is expressed on a 0-to-1 scale, where 1 indicates maximum blind-spot risk for that channel. The final MFLS correction is the equal-weight average of all four channel scores.

4. Code Implementation

BSDT is implemented as a software component called BSDTChannels, available in the public code repository (system_mode.py). It exposes three straightforward operations:

- Setup — fit(X_ref): the framework is shown a reference set of normal transactions so it learns what 'typical' looks like. This is a one-time step done before the system is deployed.
- Scoring — mfls(X): given a batch of new transactions, the system computes the Missed-Fraud Likelihood Score — a measure of how likely it is that the underlying detection model is blind to genuine fraud in those transactions.
- Output — score(X): returns the final combined value on a 0-to-1 scale, ready to be used as a correction layer on top of any existing fraud detector.

Figure 1: The production code for BSDTChannels, taken directly from the public repository. The three operations — setup, scoring, and output — correspond to the steps described above.

5. Validation Results

BSDT was first tested on its primary intended domain: detecting financial fraud. In a second stage, the same framework was applied — as an exploratory extension — to structurally similar problems in other fields, to test whether the four-channel approach had broader mathematical relevance beyond fraud detection. The results are summarised in the table below.

Dataset / Domain	Evidence	Key metric
------------------	----------	------------

Fraud benchmark (primary)	99.94% false-positive reduction	36,469 → 22 alerts
12 fraud datasets (66 variants)	Zero missed fraud in 10 of 12 datasets	Mean F1 0.787
2021 Texas ERCOT power grid (later extension)	Detected instability signals associated with the 2021 grid stress event 72 hours in advance	AUC 0.9997 (public data)
2022 Terra/Luna (later extension)	Detected on-chain anomaly signals 30 hours preceding the de-pegging event	On-chain data only
2008 US banking crisis (later extension)	Early-warning signals consistent with known institution failures	AUROC 0.867

Table 1: Validation results across five test scenarios. The first two rows are the primary validation (financial fraud detection). The final three rows show the same framework applied to structurally similar problems in other fields. All data used is publicly available.

The fraud-detection results (first two rows) are the core validation evidence. The three cross-domain rows show the same framework independently identifying early-warning signals in power-grid stress and financial crisis data. This is not because those domains were specifically targeted, but because they share the same underlying mathematical structure as fraud detection: things that look normal on the surface, unusual activity signals underneath, and reference points that become outdated over time. Only publicly available data was used throughout; no private or privileged information was accessed at any stage.

Independent validation — no data leakage: all 12 fraud benchmarks were evaluated using strict time-based splits, meaning the model was trained only on data from before the test period. A dedicated verification script (test_no_leakage_all.py) confirms that no data contamination occurred across any of the 12 datasets.

6. Reproducibility

BSDT was evaluated across 792 separate test runs: 66 model configurations, each tested on 12 independent fraud datasets. This scale rules out the possibility that a favourable result was a coincidence or a cherry-picked example. Every test script is publicly available and can be run on a standard laptop in 2–8 minutes using free Python libraries. No specialist hardware is required.

Table 2: The six test scripts covering the full 792-evaluation study. Each can be run independently by any assessor with a standard Python installation. The scale of testing means no single result can be dismissed as coincidental or cherry-picked.

Figure 2: The commands required to reproduce the results from a standard Python 3.11+ installation, run from the public code repository.

7. Technical Significance

BSDT offers three practical contributions to the field of applied machine learning — artificial intelligence systems used in real-world operational settings:

- Explains why, not just when: BSDT identifies which of the four failure mechanisms caused a specific detection to be missed — not simply that the system failed. This gives the teams managing real fraud-detection operations specific, actionable information rather than an unexplained failure signal.
- Works on top of existing systems: the correction score BSDT produces can be applied alongside any existing fraud-detection model, without replacing or retraining it. Organisations that have already invested in deployed detection infrastructure can benefit immediately.
- Broader applicability: the same four failure patterns BSDT identifies in fraud detection — camouflage, information gaps, unseen patterns, and outdated baselines — appear to recur in other fields that share similar mathematical characteristics. This broader relevance is exploratory and is not presented as a single trained system applicable to all domains.

BSDT was developed entirely independently, published openly without any institutional support, and has since been incorporated into postgraduate course reading materials by an external academic who had no prior connection to the applicant. The full publication record is documented in OC4-2; the academic adoption evidence is in OC4-3.

$\delta_C = 1 - \text{clip}(\ x - \mu_{\text{ref}}\ / d_{\text{max}}, 0, 1)$ (Camouflage) $\delta_G = (\# \text{ near-zero features}) / p$ (Feature Gap) $\delta_A = \sigma((d_{\text{Mahal}} - d_{\text{ref}}) / d_{\text{ref}})$ (Activity Anomaly) $\delta_T = \sigma(0.5 \cdot (d_{\text{kNN}} / d_{\text{kNN,ref}} - 2))$ (Temporal Novelty) $E_{\text{BS}} = \sum \delta_i^2$ (Total blind-spot energy) $MFLS = \ \nabla E_{\text{BS}}\ _F$ (Gradient-norm score) $\text{Combined} = 0.5 \cdot E_{\text{BS}} + 0.5 \cdot MFLS$ (Equal-weight average) class BSDTChannels: """Blind-Spot Detection Tensor — four complementary channels.""" def fit(self, X_ref: np.ndarray) -> "BSDTChannels": self.mu_ = X_ref.mean(axis=0) self.d_max_ = max(float(np.linalg.norm(X_ref - self.mu_, axis=1).max()), self.eps) self.mahal_ref_median_ = float(np.median(self.mahalanobis(X_ref))) self.ref_knn_median_ = float(np.median(ref_dists[:, -1])) def mfls(self, X: np.ndarray) -> np.ndarray: return np.linalg.norm(self._gradient_vectors(X), axis=1) def score(self, X: np.ndarray) -> np.ndarray: e = self.energy(X) m = self.mfls(X) return 0.5 * (e / e.max()) + 0.5 * (m / m.max())		
Script	What it tests	Result
test_no_leakage_all.py	Zero data leakage across all 12 fraud datasets	Pass — all temporal splits clean

test_mode4.py	BSDT blind-spot correction on fraud benchmarks	99.94% FP reduction, 0 missed fraud
test_mode6_bench.py	66-variant systematic comparison	Best model wins 10/12 datasets
test_molecular_bench.py	Cross-domain: molecule property prediction	BSDT improves MAE on 4/6 targets
test_geo_cyber.py	Cross-domain: cyber threat + geopolitical risk	AUC improvement vs baseline
run_bsd_t_benchmarks.py	Full 792-evaluation orchestration script	All 66 variants, 12 datasets

```
# Run full BSDT benchmark suite (792 evaluations)
# From repository root:
py -3 run_bsd_t_benchmarks.py

# Reproduce the key fraud result:
py -3 test_mode4.py

# Verify zero data leakage:
py -3 test_no_leakage_all.py

# Cross-domain: ERCOT power grid + Terra/Luna crypto:
py -3 test_geo_cyber.py
```